

Python Cheat Sheet

27 tables · 479 concepts · Updated 2026-09-27 · [cheatgrid.com](#)

Python is a **high-level, interpreted programming language** used for web development, data science, automation, machine learning and systems programming. Created by Guido van Rossum in 1991, it emphasizes readable code through significant whitespace and a philosophy of "there should be one obvious way to do it", backed by a vast package ecosystem and an active global community. Python 3.14 (released October 2025) introduced **deferred annotation evaluation**, **t-strings** and officially supported **free-threading**, one of the most significant runtime shifts in the language's history. Writing good Python is less about syntax than about knowing when a **list comprehension beats a loop**, how **duck typing** works, and why context managers matter for resource safety.

CONTENTS

Data Types	2
Operators	3
String Operations	4
Slicing and Indexing	5
List, Dict and Set Methods	6
Control Flow	7
Functions	8
Common Built-in Functions	9
Comprehensions and Generators	10
Exception Handling	11
File Operations	12
Modules and Imports	13
Virtual Environments and Packaging	13
Classes and OOP	14
Magic Methods (Dunder Methods)	16
Dataclasses and Modern Class Features	17
Enum Module	18
Decorators and functools	18
Context Managers	19
Standard Library Essentials	20
Collections Module	21
Itertools Module	22
Regular Expressions	23
Logging Module	24
Type Hints	25
Async/Await (Asyncio)	26
Concurrency	27
References	29

TABLE 1

Data Types

Python's built-in types are the foundation every program builds on. Knowing which are mutable, which are hashable, and how each behaves in memory keeps you out of the most common Python traps: assuming `bool` is *not* an `int`, or that two distinct lists with equal contents are the same object.

Type	Example	Description
int	<code>x = 42</code>	Arbitrary-precision integers , no overflow limit unlike most languages.
float	<code>y = 3.14</code>	- Double-precision floating-point (64-bit IEEE 754) beware rounding errors with exact comparisons.
str	<code>s = "hello"</code>	Immutable sequence of Unicode characters - supports f-strings, slicing, and a rich method surface.
bool	<code>flag = True</code>	Subclass of int: <code>True</code> is 1 and <code>False</code> is 0 - this surprises most newcomers
Truth Value Testing	<code>bool(0), bool(""), bool([]) # all False</code>	- Every object has a truth value used in <code>if</code> / <code>while</code> <code>None</code>, <code>False</code>, <code>0</code>, and empty sequences/collections are falsy; everything else is truthy
Mutable vs Immutable	<code>hash((1, 2)) # ok hash([1, 2]) # TypeError</code>	- Immutable types (<code>int</code>, <code>str</code>, <code>tuple</code>) can be hashed and used as dict keys or set members mutable containers like list and dict cannot
list	<code>arr = [1, 2, 3]</code>	Mutable ordered sequence , the most versatile and commonly used collection type.
dict	<code>d = {"key": "val"}</code>	- Mutable mapping of unique keys to values insertion-ordered since Python 3.7.
tuple	<code>t = (1, 2, 3)</code>	Immutable ordered sequence - hashable when elements are hashable, usable as dict keys.
set	<code>s = {1, 2, 3}</code>	- Mutable unordered collection of unique hashable elements - O(1) membership test.
frozenset	<code>fs = frozenset([1, 2])</code>	Immutable version of <code>set</code> - hashable, can be a dict key or set element.
None	<code>x = None</code>	Singleton representing absence of value - distinct from empty collections or zero
bytes	<code>b = b"data"</code>	Immutable sequence of byte values (0-255) - for binary data and network protocols
bytearray	<code>ba = bytearray(b"x")</code>	Mutable version of <code>bytes</code> , for in-place modification of binary data.
range	<code>r = range(10)</code>	Immutable sequence representing an arithmetic progression , memory-efficient for iteration.
complex	<code>z = 1 + 2j</code>	- Numbers with real and imaginary parts <code>j</code> suffix denotes the imaginary unit
Decimal / Fraction	<code>Decimal("0.1") + Decimal("0.2") # Decimal('0.3') Fraction(1, 3) + Fraction(1, 6) # Fraction(1, 2)</code>	<code>decimal.Decimal</code> gives exact base-10 arithmetic, avoiding binary float rounding errors <code>fractions.Fraction</code> stores exact rational numbers

TABLE 2

Operators

Every Python operator looks familiar from other languages, but most carry one or two Python-specific subtleties: true vs floor division, modulo whose sign follows the divisor, right-associative `**`, short-circuit `and` / `or` that return operands instead of booleans. The newer additions (`:=` walrus, `|` for dicts) reflect Python's continuing language design.

Operator	Example	Description
Arithmetic	<code>x + y</code> , <code>x - y</code> , <code>x * y</code>	Addition, subtraction, multiplication: work on numbers and some sequences (like <code>str</code> , <code>list</code>).
Division	<code>x / y</code>	Always returns a float , even for integer operands: true division since Python 3.
Floor division	<code>x // y</code>	- Rounds down to nearest integer - result type follows operands (<code>int//int → int</code>).
Modulo	<code>x % y</code>	Remainder whose sign matches the divisor , unlike C/Java where it matches the dividend.
Exponentiation	<code>x ** y</code>	- Raises <code>x</code> to power <code>y</code> right-associative and supports fractional/negative exponents
Augmented assignment	<code>lst += [4]</code> <code>x += 1</code>	- Combines a binary operation and assignment in one statement (<code>+=</code>, <code>-=</code>, <code>*=</code>, ...) mutates in place for mutable types like <code>list</code>, but rebinds to a new object for immutables like <code>int</code>
Comparison	<code>x == y</code> , <code>x < y</code> , <code>x >= y</code>	Equality and ordering, chainable like <code>1 < x < 10</code> (evaluates each pair once).
Identity	<code>x is y</code>	Tests if objects are the same instance in memory, not value equality.
Membership	<code>x in y</code>	Checks if <code>x</code> exists in container <code>y</code> $O(1)$ for sets and dicts
Logical AND	<code>x and y</code>	Returns the first falsy value , or the last value if all are truthy: short-circuits without checking <code>y</code> when unnecessary.
Logical OR	<code>x or y</code>	Returns the first truthy value , or the last value if all are falsy: short-circuits without checking <code>y</code> when unnecessary.
Logical NOT	<code>not x</code>	Boolean negation: returns <code>True</code> if <code>x</code> is falsy.
Bitwise AND	<code>x & y</code>	- Bitwise AND of integers - also set intersection for sets
Bitwise OR	<code>x y</code>	- Bitwise OR of integers - also set union for sets
Bitwise XOR	<code>x ^ y</code>	- Bitwise exclusive OR - also symmetric difference for sets
Bitwise NOT	<code>~x</code>	One's complement: inverts all bits - result is <code>-(x+1)</code>.
Left shift	<code>x << n</code>	- Shifts bits left by <code>n</code> - equivalent to multiplying by <code>2**n</code>.
Right shift	<code>x >> n</code>	- Shifts bits right by <code>n</code> - equivalent to floor-dividing by <code>2**n</code>.
Operator precedence	<code>-1 ** 2</code> <code># -1, not 1: ** binds tighter than unary -</code>	- Sets which operator runs first when several are mixed - exponentiation binds tighter than unary minus, a common gotcha
Walrus operator	<code>(x := 5)</code>	Assigns and returns a value in a single expression, useful in <code>while</code> loops and comprehensions (3.8+).

Operator	Example	Description
Dict merge operator	<code>d1 d2</code>	- Merges two dicts into a new dict <code> =</code> updates in-place (Python 3.9+).

TABLE 3

String Operations

Strings are immutable Unicode sequences with a rich method surface for searching, splitting, joining, and formatting. Modern code reaches for `f-strings` for interpolation, `str.join` for performant concatenation, and the `removeprefix` / `removesuffix` pair for safe edge trimming, always passing `encoding="utf-8"` to `open()` to avoid locale-dependent surprises.

Method	Example	Description
f-strings	<code>f"{x} + {y} = {x+y}"</code> <code>f"{x=}"</code>	Fastest and most readable formatting, supports expressions, format specs, and nested quotes since 3.12 - the <code>=</code> debug specifier prints the expression source alongside its value
Format spec mini-language	<code>f"{1234567.891:12,.2f}"</code>	- Controls width, precision, alignment, sign, and <code>,</code> / <code>_</code> digit grouping inside a format spec - shared by f-strings, <code>format()</code>, and <code>str.format()</code>
format()	<code>"{} {}".format(a, b)</code>	String formatting with positional/keyword arguments, more verbose than f-strings.
split()	<code>s.split(",")</code>	- Splits into list by delimiter default splits on any whitespace and removes empty strings
join()	<code>", ".join(items)</code>	Concatenates an iterable into a single string: preferred over <code>+=</code> in loops for performance.
splitlines()	<code>"ab c\n\nde fg".splitlines()</code>	- Splits a string into a list of lines at line boundaries (<code>\n</code>, <code>\r</code>, <code>\r\n</code>) <code>keepends=True</code> keeps the terminators
strip()	<code>s.strip()</code>	- Removes leading and trailing whitespace <code>lstrip()</code> / <code>rstrip()</code> for one side only
removeprefix() / removesuffix()	<code>s.removeprefix("pre")</code>	- Removes exact prefix or suffix if present - returns unchanged string otherwise (Python 3.9+).
replace()	<code>s.replace("old", "new")</code>	- Returns copy with all occurrences replaced - optional <code>count</code> parameter to limit replacements
upper() / lower()	<code>s.upper()</code>	- Case conversion <code>casefold()</code> for aggressive case-insensitive comparisons across locales
startswith() / endswith()	<code>s.startswith("pre")</code>	- Checks prefix/suffix accepts a tuple of multiple options
find() / index()	<code>s.find("sub")</code>	- Finds substring position <code>find()</code> returns -1 if not found, <code>index()</code> raises <code>ValueError</code>.
count()	<code>s.count("x")</code>	Returns the number of non-overlapping occurrences of a substring.
partition()	<code>s.partition(":")</code>	- Splits into 3-tuple (before, sep, after) - always returns 3 parts even if separator not found
isdigit() / isalpha()	<code>s.isdigit()</code>	- Character-type checks - many variants: <code>isalnum()</code>, <code>isspace()</code>, <code>islower()</code>, <code>isupper()</code>.
encode() / decode()	<code>s.encode("utf-8")</code>	- Converts <code>str</code> → <code>bytes</code> via encoding <code>bytes.decode("utf-8")</code> reverses it
center() / ljust() / rjust()	<code>s.center(20, "*")</code>	Pads string to a given width with fill character - useful for formatting output
zfill()	<code>"42".zfill(5)</code>	Pads with leading zeros to specified width: <code>"00042"</code> .

Method	Example	Description
title() / capitalize()	<code>s.title()</code>	<code>title()</code> capitalizes each word <code>capitalize()</code> only the first character of the whole string
translate()	<code>s.translate(str.maketrans("abc","ABC"))</code>	Replaces characters per a mapping table , efficient for multi-character substitution.
t-strings	<code>t"Hello {name}"</code>	Template strings producing a Template object for custom processing and safe interpolation (Python 3.14+).
string.Template	<code>Template("\$who likes \$what").substitute(who="tim", what="kung pao")</code>	<code>\$</code> -based substitution templates for simple, safe interpolation of untrusted input <code>safe_substitute()</code> leaves missing placeholders unchanged instead of raising
%-formatting (legacy)	<code>"%s %d" % (s, n)</code>	C-style formatting, replaced by f-strings but still seen in legacy code.

TABLE 4

Slicing and Indexing

Slice syntax `[start:stop:step]` is the unified way to index Python sequences: zero-based, half-open, and quietly forgiving of out-of-range indices. The same three components handle reversal (`[::-1]`), shallow copies (`[:]`), every-other-element selection (`:::2`), and in-place edits through slice assignment and deletion.

Pattern	Example	Description
Positive indexing	<code>s[0] , s[2]</code>	- Zero-based indexing from the start <code>s[0]</code> is the first element.
Negative indexing	<code>s[-1] , s[-2]</code>	Indexes from the end: <code>s[-1]</code> is the last element , <code>s[-2]</code> the second-to-last.
Basic slice	<code>s[1:4] , s[:3] , s[2:]</code>	<code>start:stop</code> selects up to (not including) <code>stop</code> - omitting <code>start</code> defaults to <code>0</code> , omitting <code>stop</code> defaults to the sequence's length
Reverse	<code>s[::-1]</code>	Omitting <code>start</code> and <code>stop</code> with a negative step walks the whole sequence backward.
Step slicing	<code>s>:::2</code>	- The third slice component sets the step - keeps every 2nd element
Full copy	<code>s[:]</code>	Same as <code>s.copy()</code> : makes a shallow copy of the whole sequence.
Negative step slice	<code>s[4:1:-1]</code>	With a negative step, indices count down from <code>start</code> to (not including) <code>stop</code> .
Slice assignment	<code>letters[2:5] = ['C', 'D', 'E']</code>	- Replaces the sliced range in place - a different-length replacement grows or shrinks the sequence
Slice deletion	<code>del a>:::2</code>	Removes every element the slice selects, same as assigning <code>[]</code> to that slice.
Out-of-range slicing	<code>s[10:20]</code> on a 5-item <code>s</code>	- Indexing past the end raises <code>IndexError</code> slicing past the end is forgiving and returns what exists
slice() objects	<code>idx = slice(1, 4)</code> <code>s[idx]</code>	<code>slice(start, stop, step)</code> builds a reusable slice object, the same object slicing syntax generates internally.

TABLE 5

List, Dict and Set Methods

The methods that do the daily work on lists, dicts and sets, plus the packing/unpacking and copy semantics that go with them; the types themselves are covered in **Data Types**.

Concept	Example	Description
list.append()	<pre>nums = [1, 2] nums.append(3)</pre>	Adds a single value to the end of the list in place.
list.extend()	<pre>nums.extend([4, 5])</pre>	- Adds every item from an iterable to the end <code>append([4, 5])</code> would nest it as one item instead
list.pop()	<pre>stack = [1, 2, 3] stack.pop()</pre>	- Removes and returns the item at <code>index</code> (default the last) <code>pop(0)</code> is slow on a long list; use <code>collections.deque</code> for a queue
list.insert()	<pre>nums.insert(0, 'x')</pre>	Inserts a value at the given index, shifting later items right.
list.remove()	<pre>nums.remove(3)</pre>	- Removes the first item equal to value - raises <code>ValueError</code> if none matches
list.index()	<pre>nums.index(3)</pre>	- Returns the index of the first matching value - raises <code>ValueError</code> if absent
list.count()	<pre>nums.count(3)</pre>	Returns how many times a value appears in the list.
list.sort(key=, reverse=)	<pre>people.sort(key=lambda p: p.age, reverse=True)</pre>	- Sorts the list in place - the builtin <code>sorted()</code> does the same comparison but returns a new list
del statement	<pre>del a[0] del a[2:4]</pre>	- Removes an item, a slice, or an entire variable - unlike <code>pop()</code> it returns nothing
dict.get()	<pre>config.get('timeout', 30)</pre>	Returns the value for a key, or <code>default</code> (None by default) instead of raising <code>KeyError</code> .
dict.setdefault()	<pre>groups.setdefault(key, []).append(item)</pre>	Returns the key's value if present, otherwise inserts it with <code>default</code> and returns that.
dict.update()	<pre>d.update(other, x=1)</pre>	Merges another mapping or keyword arguments into the dict in place, overwriting existing keys.
dict.pop()	<pre>d.pop('key', None)</pre>	- Removes a key and returns its value, or <code>default</code> if missing - raises <code>KeyError</code> with no default given
dict.items() / .keys() / .values()	<pre>for k, v in d.items(): print(k, v)</pre>	Return live views onto the dict's pairs, keys or values that update as the dict changes.
Dict mutation while iterating	<pre>for k in list(d): if drop(k): del d[k]</pre>	- Adding or deleting keys while iterating the dict or a view can raise <code>RuntimeError</code> - iterate over <code>list(d)</code> instead
set.add()	<pre>s.add('x')</pre>	Adds a single element to the set.
set.discard() vs set.remove()	<pre>s.discard('x') s.remove('y')</pre>	- Both remove an element <code>discard</code> stays silent if it is missing, <code>remove</code> raises <code>KeyError</code>.
Set union (/ .union())	<pre>a b</pre>	Returns a new set with every element from both sets.
Set intersection (& / .intersection())	<pre>a & b</pre>	Returns a new set with only the elements common to both sets.
Set difference (- / .difference())	<pre>a - b</pre>	Returns a new set with the elements of <code>a</code> that are not in <code>b</code> .
set.issubset() / issuperset()	<pre>{1, 2}.issubset({1, 2, 3})</pre>	Tests whether every element of one set is contained in (or contains) the other.

Concept	Example	Description
Tuple packing and unpacking	<pre>t = 12345, 54321, 'hello!' x, y, z = t</pre>	- Comma-separated values are packed into a tuple - assigning it to several names unpacks it back
Star (extended) unpacking	<pre>first, *rest = [1, 2, 3, 4]</pre>	One starred target collects the remaining items into a list, wherever it sits among the targets.
Shallow copy	<pre>b = a.copy()</pre>	Builds a new outer container that still shares references to the same nested objects as the original.
copy.deepcopy()	<pre>import copy b = copy.deepcopy(a)</pre>	Recursively copies nested objects too, so the copy shares no mutable state with the original.

TABLE 6

Control Flow

Python's control-flow keywords stay deliberately small: `if` / `elif` / `else` driven by truthiness, two loop forms with the unusual `else` clause, and structural `match` with rich pattern types (Python 3.10 and later). The `match` statement goes beyond a simple switch: it deconstructs sequences, mappings and class instances, and supports OR patterns, guards, as-patterns and wildcard captures in one readable block.

Statement	Example	Description
if	<pre>if x > 0: print("pos")</pre>	Conditional execution based on the truthiness of an expression.
elif	<pre>elif x == 0: print("zero")</pre>	- Chains multiple conditions - only the first true branch executes
else	<pre>else: print("neg")</pre>	Executes when all preceding conditions are false .
Ternary expression	<pre>x if cond else y</pre>	Inline conditional: returns <code>x</code> if cond is true , otherwise <code>y</code> .
for loop	<pre>for i in range(5): print(i)</pre>	- Iterates over any iterable object - the most common loop type
while loop	<pre>while x > 0: x -= 1</pre>	- Repeats while condition is truthy - check happens before each iteration
break	<pre>break</pre>	Exits innermost enclosing loop immediately.
continue	<pre>continue</pre>	Skips to the next iteration of the innermost loop.
else on loops	<pre>for i in items: pass else: print("done")</pre>	Executes if the loop completes without hitting break , useful for search patterns.
pass	<pre>pass</pre>	Null operation: a syntactic placeholder where a statement is required.
match-case (literal)	<pre>match x: case 1: print("one")</pre>	Structural pattern matching that compares the subject against literal values .
match wildcard	<pre>case _:</pre>	Catch-all pattern that matches anything, similar to <code>default</code> in other languages.
match sequence pattern	<pre>case [first, *rest]:</pre>	- Matches and unpacks sequences <code>*rest</code> captures the remaining items
match mapping pattern	<pre>case {"action": a}:</pre>	- Matches dicts by key presence <code>**rest</code> captures the remaining keys
match class pattern	<pre>case Point(x, y): print(x, y)</pre>	- Deconstructs class instances by keyword attribute (<code>Point(x=x, y=y)</code>) - positional args match via <code>__match_args__</code>

Statement	Example	Description
match OR pattern	<code>case 1 2 3:</code>	• Matches multiple values in one case arm using <code> </code> .
match guard	<code>case x if x > 0:</code>	• Adds a condition to a pattern • the branch runs only if the guard evaluates true
match as-pattern	<code>case [x, y] as point:</code>	Binds the entire matched value to a name in addition to matching the pattern on its left.

TABLE 7

Functions

Functions are first-class values in Python: they can be passed around, returned from other functions, given default values, and unpacked from iterables. Two rough edges trip up most practitioners: a default argument is evaluated once, at definition time, and the `/` and `*` separators in a signature enforce positional-only and keyword-only calling conventions.

Technique	Example	Description
Function definition	<pre>def func(a, b): return a + b</pre>	- Creates a reusable code block <code>return</code> without a value returns <code>None</code> .
Default arguments	<pre>def f(x, y=10):</pre>	- Parameters with default values, must come after required params - evaluated once at definition.
Mutable default argument trap	<pre>def f(a, L=[]): L.append(a) return L f(1); f(2) # [1, 2], same list reused</pre>	- A mutable default is created once and shared across calls - use <code>L=None</code> and create the list inside the body instead
Keyword arguments	<pre>f(x=1, y=2)</pre>	Named parameters passed as <code>name=value</code> , allowing any order at the call site.
Return multiple values as a tuple	<pre>def func1(a, b): a = 'new-value' b = b + 1 return a, b x, y = func1('old-value', 99)</pre>	<code>return a, b</code> packs the results into a tuple , unpacked by the caller.
*args	<pre>def f(*args):</pre>	Variable positional arguments, collected as a tuple .
kwargs	<pre>def f(kwargs):</pre>	Variable keyword arguments, collected as a dict .
Unpacking in calls	<pre>f(*args, **kwargs)</pre>	Expands sequences and mappings into function arguments at call time.
Lambda	<pre>lambda x: x * 2</pre>	Anonymous inline function, a single expression only, no statements.
Docstring	<pre>"""Description."""</pre>	First string in a function body, accessible via <code>__doc__</code> and used by <code>help()</code> .
Positional-only parameters	<pre>def f(a, b, /, c):</pre>	Parameters before <code>/</code> cannot be passed by keyword , enforcing positional calling (Python 3.8+).
Keyword-only parameters	<pre>def f(a, *, b):</pre>	Parameters after a bare <code>*</code> must be passed by keyword , preventing accidental positional misuse.
Global and nonlocal scope	<pre>def outer(): x = 0 def inner(): nonlocal x x += 1 inner(); return x</pre>	- Name lookup follows local, enclosing, global, built-in (LEGB) <code>global</code> / <code>nonlocal</code> allow rebinding an outer name.

Technique	Example	Description
Closure	<pre>def linear(a, b): def result(x): return a * x + b return result</pre>	An inner function that remembers the free variables of its enclosing scope after that scope has returned.

TABLE 8

Common Built-in Functions

Python's built-in functions turn up in nearly every program: `len`, `enumerate`, `zip`, `map`, `filter`, `sum`, `sorted`, `min` / `max`, `all` / `any`, `isinstance`. Mastering their `key=` arguments, lazy-iterator returns, and edge cases (vacuous `all([])`, `zip` truncation, `input` always returning `str`) keeps day-to-day code short and correct.

Function	Example	Description
len()	<code>len([1, 2, 3])</code>	Returns the number of items in a sequence or collection, via <code>__len__</code> .
int() / float() / str()	<pre>int('FACE', 16) → 64206 float('3.14') , str(3.14)</pre>	- Converts a value to <code>int</code>, <code>float</code> or <code>str</code> <code>int()</code> accepts an optional <code>base</code> for string input
enumerate()	<code>enumerate(items, start=0)</code>	Adds a counter to an iterable, returning (index, value) pairs.
zip()	<code>zip(list1, list2)</code>	- Pairs elements from iterables, stopping at the shortest input <code>strict=True</code> raises if lengths differ (3.10+)
map()	<code>map(func, iterable)</code>	Applies a function to each element, returning a lazy iterator .
filter()	<code>filter(predicate, items)</code>	Keeps elements where the predicate is truthy , as a lazy iterator.
sorted()	<code>sorted(items, key=func)</code>	- Returns a new sorted list without modifying the original <code>reverse=True</code> for descending
sum()	<code>sum(numbers, start=0)</code>	Sums a numeric iterable, with an optional starting value .
round()	<pre>round(2.5) → 2 round(1.5) → 2</pre>	- Rounds to <code>ndigits</code> precision (default whole number) - ties round to the nearest even choice (banker's rounding)
min() / max()	<code>min(items, key=func)</code>	Returns the smallest/largest element, with an optional key function .
all()	<code>all([True, True, False])</code>	Returns <code>True</code> if all elements are truthy or the iterable is empty (vacuously true).
any()	<code>any([False, False, True])</code>	Returns <code>True</code> if any element is truthy .
reversed()	<code>reversed(seq)</code>	Returns a reverse iterator without modifying the original.
isinstance()	<code>isinstance(obj, type)</code>	- Checks an object's type, accepting a tuple of types - works with inheritance
type()	<code>type(obj)</code>	- Returns an object's exact type - For type checks, prefer <code>isinstance()</code>.
getattr() / setattr() / hasattr()	<code>getattr(obj, "name", default)</code>	- Dynamic attribute access <code>getattr</code> supports a default to avoid <code>AttributeError</code>.
vars() / dir()	<code>vars(obj)</code>	<code>vars()</code> returns <code>__dict__</code> <code>dir()</code> lists all attributes and methods of an object
abs() / divmod()	<code>abs(-5)</code> , <code>divmod(17, 5)</code>	<code>abs()</code> returns magnitude <code>divmod(a, b)</code> returns (quotient, remainder) as a tuple
hex() / oct() / bin()	<code>hex(255) → "0xff"</code>	Converts integers to hexadecimal/octal/binary string representations.
ord() / chr()	<pre>ord('a') → 97 chr(97) → 'a'</pre>	Converts between a single-character string and its Unicode code point (<code>chr</code> is the inverse of <code>ord</code>).
callable()	<code>callable(func)</code>	Returns <code>True</code> if object appears callable (has <code>__call__</code>).

Function	Example	Description
<code>hash()</code>	<code>hash("hello")</code>	- Returns integer hash of a hashable object - consistent within a process
<code>id()</code>	<code>id(obj)</code>	Returns an object's unique identifier , guaranteed constant for its lifetime.
<code>iter() / next()</code>	<code>it = iter([1, 2])</code> <code>next(it, None)</code>	<code>iter()</code> returns an iterator from an iterable <code>next()</code> advances it, returning default instead of raising <code>StopIteration</code>
<code>input()</code>	<code>input("Enter: ")</code>	Reads a line from stdin, always returning a string .
<code>print()</code>	<code>print(*values, sep=" ", end="\n")</code>	Writes to stdout, with a customizable separator and line ending .
<code>breakpoint()</code>	<code>breakpoint()</code>	Drops into the debugger at the call site (<code>pdb.set_trace()</code> by default).
<code>help()</code>	<code>help(str)</code>	Prints interactive help for a module, function, class, method or keyword.

TABLE 9

Comprehensions and Generators

List, dict, and set comprehensions build a collection from an iterable in one expression, typically faster and easier to read than the equivalent `for / append` loop. Generator expressions and `yield`-based functions use almost the same syntax but produce lazy iterators, which is what lets Python stream gigabytes without loading them into memory.

Technique	Example	Description
List comprehension	<code>[x**2 for x in range(10)]</code>	Creates a list from an iterable in one expression: faster and cleaner than a for-loop with append.
With condition	<code>[x for x in nums if x > 0]</code>	Filters elements: the <code>if</code> clause acts as an inline filter on the iterable.
Conditional expression in comprehension	<code>['even' if x % 2 == 0 else 'odd' for x in nums]</code>	- Maps every element through <code>x if cond else y</code> instead of dropping it different from the filtering if above, which removes elements
Dict comprehension	<code>{k: v**2 for k, v in d.items()}</code>	Creates a dict from an iterable: key-value pairs with unique keys (later values win).
Set comprehension	<code>{x**2 for x in nums}</code>	Creates a set: automatically deduplicates results.
Nested comprehension	<code>[x+y for x in a for y in b]</code>	Multiple <code>for</code> clauses in one comprehension: left-to-right nesting, outer loop first.
Generator expression	<code>(x**2 for x in range(10))</code>	- Lazy evaluation, memory-efficient for large sequences - values are not stored, only computed as consumed
Generator function	<code>def gen():</code> <code>yield x</code>	Function with <code>yield</code> : pauses and resumes execution on each <code>next()</code> call.
yield from	<code>yield from iterable</code>	Delegates to a subiterator: simplifies nested generators (Python 3.3+).
Generator pipeline	<code>evens = (x for x in data</code> <code>if x%2==0)</code> <code>result = sum(evens)</code>	- Chains generators lazily, never materializes an intermediate list - memory-efficient for large datasets
Generator send() / close()	<code>g = counter(10)</code> <code>next(g)</code> <code>g.send(8)</code> <code>g.close()</code>	<code>send(value)</code> resumes the generator and becomes the result of the paused <code>yield</code> <code>close()</code> raises <code>GeneratorExit</code> inside it to stop the iteration
Exhausted generator (StopIteration)	<code>g = (x for x in range(3))</code> <code>list(g)</code> <code>list(g)</code>	A generator runs only once : once it raises <code>StopIteration</code> it stays empty, so a second pass needs a fresh generator.

TABLE 10

Exception Handling

Exceptions are Python's primary error-signalling mechanism, built on a four-clause structure (`try` / `except` / `else` / `finally`) with clean chaining via `raise ... from ...`. `ExceptionGroup` and `except*` (3.11+) let code express multiple concurrent failures, essential for `asyncio.TaskGroup` and structured-concurrency designs.

Technique	Example	Description
try-except	<pre>try: risky() except ValueError:</pre>	- Runs the try block and jumps to a matching except on error - list specific types before general ones
Multiple exceptions	<pre>except (ValueError, KeyError):</pre>	Catches several types in one clause using a tuple of exception classes .
Bracketless multiple exceptions (3.14)	<pre>except TimeoutError, ConnectionRefusedError:</pre>	Python 3.14 lets the parentheses be dropped for multiple types when no <code>as</code> clause is used (PEP 758).
except with variable	<pre>except ValueError as e:</pre>	- Captures the exception object access message and attributes via <code>e</code>.
else clause	<pre>else: print("success")</pre>	Runs only when the <code>try</code> block raises no exception , before <code>finally</code> .
finally	<pre>finally: cleanup()</pre>	Always executes, even if an exception is raised or <code>return</code> is hit - used for resource cleanup
return/break/continue in finally (avoid)	<pre>try: return True finally: return False</pre>	- A <code>return</code>, <code>break</code> or <code>continue</code> inside <code>finally</code> silently swallows the try block's outcome - Python 3.14 flags it with a <code>SyntaxWarning</code> (PEP 765).
raise	<pre>raise ValueError("msg")</pre>	- Throws an exception - a bare <code>raise</code> re-raises the current active exception
Exception chaining	<pre>raise NewError() from e</pre>	Chains a new exception to the one that caused it, preserving the original traceback for debugging.
Suppress chaining (from None)	<pre>except OSError: raise RuntimeError from None</pre>	Disables automatic exception chaining so the traceback shows only the new exception .
Custom exceptions	<pre>class MyError(Exception):</pre>	- Defines a new exception type - inherit from <code>Exception</code> or a more specific subclass
Built-in exception hierarchy	<pre>except LookupError: # catches IndexError, KeyError</pre>	<code>ValueError</code>, <code>KeyError</code>, <code>TypeError</code> and <code>OSError</code> sit under shared base classes - catching the base catches all its subclasses
add_note()	<pre>except Exception as e: e.add_note("context") raise</pre>	- Attaches a note to a caught exception - the traceback prints every note after the exception
ExceptionGroup	<pre>raise ExceptionGroup("msg", [ValueError(), KeyError()])</pre>	- Groups multiple exceptions together - handled with <code>except*</code> (Python 3.11+).
except*	<pre>except* ValueError as eg:</pre>	- Catches matching exceptions from an ExceptionGroup - multiple <code>except*</code> clauses can each match (Python 3.11+).
Bare except (avoid)	<pre>except:</pre>	- Matches with no exception type, so it catches everything including <code>KeyboardInterrupt</code> - hides bugs, avoid it

TABLE 11

File Operations

File I/O uses a unified `open()` API parameterized by mode (`r` / `w` / `a` / `b` / `+`) and is best paired with `with` to guarantee cleanup on every exit path. Modern code increasingly leans on `pathlib.Path` for path manipulation and one-call `read_text` / `write_text` shortcuts, and remembers to pass `encoding="utf-8"` to dodge locale-dependent surprises across operating systems.

Technique	Example	Description
with statement	<code>with open("f.txt") as f:</code>	Context manager that auto-closes the file even if an exception occurs.
open() for reading	<code>f = open("f.txt", encoding="utf-8")</code>	Always specify <code>encoding="utf-8"</code> to avoid locale-dependent surprises on Windows.
Mode "r" (default)	<code>open("f.txt", "r")</code>	Read mode, raising <code>FileNotFoundError</code> if the file does not exist.
Mode "w"	<code>open("f.txt", "w")</code>	Write mode: truncates an existing file or creates a new one.
Mode "a"	<code>open("f.txt", "a")</code>	- Append mode: adds to the end of an existing file - creates the file if absent
Mode "x"	<code>open("f.txt", "x")</code>	Exclusive creation: raises <code>FileExistsError</code> if the file already exists .
Mode "b"	<code>open("f.bin", "rb")</code>	- Binary mode: for non-text files - returns <code>bytes</code> objects
read()	<code>content = f.read()</code>	Reads the entire file as a string, memory-intensive for large files.
Iterate file	<code>for line in f:</code>	Memory-efficient line iteration, preferred over <code>readlines()</code> for large files.
readline()	<code>line = f.readline()</code>	Reads a single line, including the newline character.
readlines()	<code>lines = f.readlines()</code>	Returns a list of all lines, with newlines preserved in each element.
write()	<code>f.write("text")</code>	Writes a string to the file, without adding a newline automatically.
writelines()	<code>f.writelines(lines)</code>	Writes a sequence of strings, without adding newlines automatically.
newline=""	<code>open("data.csv", newline="")</code>	- Controls line-ending translation in text mode <code>""</code> disables translation, needed before handing the file to <code>csv</code>
pathlib.Path	<code>Path("f.txt").read_text(encoding="utf-8")</code>	- Object-oriented path API, cleaner than <code>os.path</code> <code>copy()</code> / <code>move()</code> added in 3.14
Path.read_text() / write_text()	<code>Path("f.txt").write_text("hi", encoding="utf-8")</code>	Opens, reads or writes, and closes the file in a single call.
Path.exists() / is_file()	<code>Path("f.txt").exists()</code>	Checks whether a path exists, or points to a regular file, without opening it.
Path.iterdir()	<code>for p in Path(".").iterdir(): print(p)</code>	Yields each entry of a directory as a <code>Path</code> object, in arbitrary order.
Path.glob()	<code>sorted(Path(".").glob("*.py"))</code>	- Yields paths matching a glob pattern - use <code>**</code> to recurse into subdirectories
Path.mkdir(parents=True, exist_ok=True)	<code>Path("a/b").mkdir(parents=True, exist_ok=True)</code>	Creates a directory, also creating missing parents and not erroring if it already exists.
tempfile.NamedTemporaryFile	<code>with tempfile.NamedTemporaryFile() as f: f.write(b"data")</code>	Creates a temporary file with automatic cleanup, usable as a context manager.

TABLE 12

Modules and Imports

Python's import system stitches code into namespaces using `sys.path` (search list), `sys.modules` (cache), and a finder/loader stack you can extend. Idiomatic code prefers explicit `import` over `from X import *`, uses relative imports inside packages, and guards script-only code behind `if __name__ == "__main__":` so the same file works as both a library and a script.

Technique	Example	Description
import module	<code>import math</code>	- Imports the whole module - access members through <code>math.func()</code> .
from import	<code>from math import sqrt</code>	Imports specific names directly, no module prefix needed.
import as	<code>import numpy as np</code>	Aliases a module to a shorter name for convenience.
if name == "main"	<code>if __name__ == "__main__":</code>	Checks whether the file is run directly rather than imported, and guards script-only code.
init.py	<code>pkg/__init__.py</code>	- Marks a directory as a package - can run initialization code and define <code>__all__</code> .
Relative imports	<code>from . import sibling</code>	- Imports siblings within a package using leading dots only works inside a package, never in the main script
python -m module	<code>python -m http.server 8000</code>	Locates a module (or a package's <code>__main__</code>) via the normal import mechanism and runs it, prepending the current directory to <code>sys.path</code> .
sys.path	<code>sys.path.append(dir)</code>	List of search locations for modules, initialized from <code>PYTHONPATH</code> and the installation defaults.
importlib	<code>importlib.import_module(name)</code>	Imports a module by string name at runtime, resolving absolute or relative paths.
importlib.reload	<code>importlib.reload(module)</code>	Re-executes an already-imported module's code, useful for trying edits without restarting.
all	<code>__all__ = ["echo", "surround"]</code>	Defines the list of names a <code>from package import *</code> should bring in.
main.py	<code>python -m bandclass</code> <code># runs</code> <code>bandclass/__main__.py</code>	Gives a package a command-line entry point, run when the package itself is invoked with <code>-m</code> .
from import * (avoid)	<code>from math import *</code>	- Imports all public names into the current namespace frowned upon: it hides where a name came from and often makes code harder to read

TABLE 13

Virtual Environments and Packaging

Isolation is the first principle: one project, one environment, so dependency versions never interfere. `venv` is the built-in option (zero install, always available), while `uv` (Astral) is a fast Rust-based alternative that manages both environments *and* Python installations with near-instant dependency resolution.

Tool	Example	Description
venv create	<code>python -m venv .venv</code>	Creates a virtual environment : installs pip and copies or symlinks the interpreter.
venv activate	<code>source .venv/bin/activate</code> <code>.venv\Scripts\activate</code>	Activates the environment: modifies <code>PATH</code> so <code>python</code> / <code>pip</code> point to the venv.
pip install	<code>pip install requests</code> <code>pip install -r requirements.txt</code>	- Installs packages <code>-r</code> installs from a requirements file

Tool	Example	Description
python -m pip	<code>python -m pip install requests</code>	Runs pip through the chosen interpreter , avoiding whichever <code>pip</code> happens to be first on <code>PATH</code> .
pip install -e (editable)	<code>python -m pip install -e .</code>	Installs a local project in editable mode so source edits take effect without reinstalling.
requirements.txt	<code>python -m pip install -r requirements.txt</code>	• Plain list of packages for <code>pip install -r</code> • only a true lockfile when generated by <code>pip freeze</code> , since hand-written entries aren't pinned to transitive versions
pip freeze	<code>pip freeze > requirements.txt</code>	Snapshots current installed packages with pinned versions.
uv init	<code>uv init example</code>	Creates a new project with a <code>pyproject.toml</code> file and starter layout.
uv add	<code>uv add requests</code>	Adds a dependency to <code>pyproject.toml</code> and updates the lockfile .
uv run	<code>uv run script.py</code>	• Runs a script or command in the project's environment, creating it automatically • no manual activation needed
uv sync	<code>uv sync</code>	Installs dependencies from the lockfile , reproducing the exact environment.
uv lock	<code>uv lock</code>	Creates or updates <code>uv.lock</code> , the project's lockfile , from <code>pyproject.toml</code> .
uvx (uv tool run)	<code>uvx ruff check .</code>	Runs a CLI tool from a package in a temporary, isolated environment without adding it to the project.
uv python install	<code>uv python install 3.12</code>	Downloads and installs a specific Python version managed by uv.
pyproject.toml	<code>[project] name = "myapp" requires-python = ">=3.11"</code>	• Modern packaging standard: replaces <code>setup.py</code> • supported by pip, uv, poetry, flit
pyenv	<code>pyenv install 3.11.5 pyenv local 3.11.5</code>	• Manages multiple Python versions side by side • sets a <code>.python-version</code> file per directory
poetry	<code>poetry new project poetry add requests</code>	• Package manager with lockfile and publishing support • <code>poetry install</code> reproduces the environment

TABLE 14

Classes and OOP

Python's class model favors flexibility over enforcement: no private attributes, no compile-time type checks, no formal interfaces. What you do get is a predictable MRO via C3 linearization, descriptor-driven property access, real multiple inheritance, and `abc.ABC` for defining enforced interfaces without metaclass boilerplate.

Concept	Example	Description
Class definition	<code>class Dog: pass</code>	Creates a new type, using PascalCase naming by convention.
init	<code>def __init__(self, name):</code>	Constructor method that initializes the instance after it is created.
Instance attributes	<code>self.name = name</code>	Per-object data, defined in <code>__init__</code> or other methods via <code>self</code> .
Class attributes	<code>class Dog: species = "Canis"</code>	Shared across all instances, defined at class level .
Attribute lookup order	<code>class Warehouse: region = 'west' w = Warehouse() w.region = 'east'</code>	• Assigning to an instance attribute creates a new instance attribute with the same name that shadows the class attribute • other instances still see the class value
Methods	<code>def bark(self):</code>	Functions bound to an instance, with <code>self</code> as the first parameter by convention.

Concept	Example	Description
Inheritance	<code>class Puppy(Dog):</code>	Derives from a parent class, inheriting attributes and methods .
super()	<code>super().__init__(name)</code>	Calls a parent class method, resolving multiple inheritance correctly via the MRO.
Multiple inheritance	<code>class C(A, B):</code>	Inherits from multiple classes, using C3 MRO to resolve method order.
mro	<code>Dog.__mro__</code> (<class 'Dog'>, <class 'Animal'>, <class 'object'>)	Read-only tuple of classes considered, in order, when looking up an attribute or method .
Name mangling	<code>class Mapping:</code> <code>def __init__(self):</code> <code>self.__id = 1</code> <code># accessed as obj._Mapping__id</code>	- A single leading underscore (<code>_x</code>) marks a name as non-public by convention - a double leading underscore (<code>__x</code>) is rewritten to <code>_ClassName__x</code> to avoid subclass name clashes
@property	<code>@property</code> <code>def age(self):</code>	Getter method, accessed like an attribute but computed dynamically.
@property.setter	<code>@age.setter</code> <code>def age(self, val):</code>	Setter method that allows validation and side effects on attribute assignment.
@classmethod	<code>@classmethod</code> <code>def factory(cls):</code>	Method receives the class, not the instance , for alternative constructors.
@staticmethod	<code>@staticmethod</code> <code>def util():</code>	Regular function in the class namespace, with no implicit first parameter .
ABC / @abstractmethod	<code>from abc import ABC, abstractmethod</code> <code>class Shape(ABC):</code> <code>@abstractmethod</code> <code>def area(self): ...</code>	Enforces an interface: subclasses cannot be instantiated until every abstract method is implemented.
slots	<code>__slots__ = ('x', 'y')</code>	Declares fixed attributes, reduces memory use and blocks <code>__dict__</code> creation.
new	<code>def __new__(cls):</code>	- Static method that creates the instance before <code>__init__</code> - needed for immutable subclasses
init_subclass	<code>def __init_subclass__(cls):</code>	Hook called when a class is subclassed, a lightweight alternative to metaclasses (Python 3.6+).
Descriptor protocol	<code>def __get__(self, obj, type):</code>	Implements per-attribute access, the foundation for <code>property</code> , <code>classmethod</code> , <code>staticmethod</code> .
Metaclass	<code>class Meta(type):</code> <code>def __new__(mcs, ...):</code>	- Controls class creation itself - explored uses include interface checking, automatic property creation and frameworks such as ORMs
Singleton pattern	<code>class Logger(object):</code> <code>_instance = None</code> <code>def __new__(cls):</code> <code>if cls._instance is None:</code> <code>cls._instance =</code> <code>super().__new__(cls)</code> <code>return cls._instance</code>	Ensures only one instance exists by returning the cached object from <code>__new__</code> on later calls.

TABLE 15

Magic Methods (Dunder Methods)

Dunder methods are how custom classes plug into Python's syntax: `len()` , `obj[key]` , `for x in obj` , `==` , `+` , even calling an instance like a function. Implementing the right small set turns a class from useful to feeling built-in, and the data model documents every hook the language consults.

Method	Example	Description
repr	<code>def __repr__(self):</code>	• Official string representation, called by <code>repr()</code> • should look like valid Python when possible
str	<code>def __str__(self):</code>	• Informal string representation, used by <code>str()</code> and <code>print()</code> • defaults to <code>__repr__()</code> if not defined
eq	<code>def __eq__(self, other):</code>	• Equality operator <code>==</code> , default compares identity • defining it disables <code>__hash__</code> by default
hash	<code>def __hash__(self):</code>	• Makes instance hashable, required to use as dict key or set element • must be consistent with <code>__eq__</code> .
len	<code>def __len__(self):</code>	• Returns length, called by <code>len()</code> • must return non-negative int.
getitem	<code>def __getitem__(self, key):</code>	Indexing operator <code>[]</code> , enables subscript access .
setitem	<code>def __setitem__(self, k, v):</code>	Item assignment <code>[k] = v</code> , makes object a mutable container .
delitem	<code>def __delitem__(self, key):</code>	Deletion via <code>del obj[key]</code> , required for full mutable container protocol.
iter	<code>def __iter__(self):</code>	Returns an iterator, enables for-loop support and unpacking.
next	<code>def __next__(self):</code>	Gets next value, raises <code>StopIteration</code> when exhausted.
contains	<code>def __contains__(self, item):</code>	Membership test <code>in</code> , should return boolean .
call	<code>def __call__(self):</code>	Makes instance callable like a function, enables <code>obj()</code> syntax .
add	<code>def __add__(self, other):</code>	• Addition operator <code>+</code> , return a new object • don't modify <code>self</code> .
radd	<code>def __radd__(self, other):</code>	Reflected addition, called with swapped operands when <code>5 + obj</code> fails on the left operand's own <code>__add__</code> .
iadd	<code>def __iadd__(self, other):</code>	In-place addition for <code>+=</code> , should modify and return self when possible.
lt / gt	<code>def __lt__(self, other):</code>	Comparison operators <code><</code> and <code>></code> , enables sorting .
bool	<code>def __bool__(self):</code>	• Truthiness check, called by <code>bool()</code> and in boolean contexts • falls back to <code>__len__</code> .
getattr / getattribute	<code>def __getattr__(self, name):</code>	• <code>__getattr__</code> runs only when normal lookup fails with <code>AttributeError</code> • <code>__getattribute__</code> runs on every attribute access
format	<code>def __format__(self, spec):</code>	Custom format spec support, the hook behind <code>format()</code> and f-string <code>{obj:spec}</code> syntax.
class_getitem	<code>def __class_getitem__(cls, item):</code>	Enables generic alias syntax <code>MyClass[T]</code> , supports runtime type parameterization.

TABLE 16

Dataclasses and Modern Class Features

`@dataclass` (Python 3.7+) generates `__init__`, `__repr__`, and `__eq__` from type-annotated attributes, with knobs for `frozen`, `slots`, and `kw_only` to fit specific use cases. For runtime validation and coercion at system boundaries, the ecosystem leans on `pydantic`; for richer feature sets without the validation cost, on `attrs`.

Technique	Example	Description
@dataclass	<pre>@dataclass class Point: x: float y: float</pre>	Generates <code>__init__</code> , <code>__repr__</code> and <code>__eq__</code> from type-annotated attributes, for less boilerplate .
Field with default	<pre>x: int = 0</pre>	- Sets a default value - a mutable default (list, dict) must use <code>field(default_factory=...)</code> instead
field()	<pre>items: list = field(default_factory=list)</pre>	Customizes a field's default, repr, comparison or metadata, avoiding shared mutable defaults across instances.
order=True	<pre>@dataclass(order=True) class Point: x: int y: int</pre>	Generates <code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> and <code>__ge__</code> , comparing instances field by field as a tuple (default <code>False</code>).
frozen=True	<pre>@dataclass(frozen=True)</pre>	- Makes instances immutable - combined with the default <code>eq=True</code>, this also enables hashing for use as dict keys
slots=True	<pre>@dataclass(slots=True)</pre>	Generates <code>__slots__</code> for reduced memory use and blocks dynamic attribute creation (Python 3.10+).
kw_only=True	<pre>@dataclass(kw_only=True)</pre>	Forces every field to be keyword-only in <code>__init__</code> , preventing positional mistakes (Python 3.10+).
KW_ONLY	<pre>@dataclass class Point: x: float _: KW_ONLY y: float z: float</pre>	Pseudo-field sentinel marking every field after it as keyword-only, without affecting the fields before it (Python 3.10+).
post_init	<pre>def __post_init__(self):</pre>	Runs right after <code>__init__</code> , for computed fields or validation.
InitVar	<pre>temp: InitVar[int]</pre>	Declares an init-only pseudo-field: passed to <code>__init__</code> and <code>__post_init__</code> , but never stored on the instance.
asdict() / astuple()	<pre>asdict(obj)</pre>	Recursively converts a dataclass instance to a dict or tuple, deep-copying nested dataclasses, lists and dicts.
dataclasses.replace()	<pre>p2 = replace(p, y=5)</pre>	Creates a new instance of the same type with the given fields replaced , re-running <code>__init__</code> and <code>__post_init__</code> .
attrs	<pre>@define class Coordinates: x: int y: int</pre>	Third-party alternative to dataclasses with validators, converters, and slots by default .
Pydantic	<pre>class User(BaseModel): id: int name: str = 'John Doe'</pre>	Validates and coerces data at runtime from type hints, and can emit a JSON Schema .

TABLE 17

Enum Module

The `enum` module defines named constant sets with distinct identity, preventing typos and magic-string comparisons. Python 3.11 added `StrEnum` and made `IntEnum` / `StrEnum` `auto()` produce a lowercased member name.

Type	Example	Description
Enum	<pre>class Color(Enum): RED = 1 GREEN = 2</pre>	Base class for enumerations, members have unique identity and are accessed as <code>Color.RED</code> .
auto()	<pre>RED = auto()</pre>	Automatically assigns values: defaults to sequential integers starting at 1.
IntEnum	<pre>class Status(IntEnum): OK = 200</pre>	Enum whose members are also <code>int</code> , compatible with integer APIs.
StrEnum	<pre>class Direction(StrEnum): NORTH = auto()</pre>	Members are also <code>str</code> , <code>auto()</code> generates the lowercased member name (Python 3.11+).
Flag	<pre>class Perm(Flag): READ = auto() WRITE = auto()</pre>	- Members support bitwise operations: combine with <code> </code> - test membership with <code>in</code>
IntFlag	<pre>class Mode(IntFlag): R = 4 W = 2</pre>	Flag plus int compatibility, members behave as integers for system calls and bitmasks.
Enum iteration	<pre>list(Color)</pre>	Enumerates all members in definition order.
Enum lookup	<pre>Color(1), Color["RED"]</pre>	Access by value (<code>Color(1)</code>) or by name (<code>Color["RED"]</code>).
member.value / .name	<pre>Color.RED.value, Color.RED.name</pre>	Access the raw value or the member name string .
@unique	<pre>@unique class Mistake(Enum): ONE = 1 TWO = 1</pre>	Class decorator that raises <code>ValueError</code> if two members share a value (<code>@verify(UNIQUE)</code> generalizes this in 3.11+).

TABLE 18

Decorators and functools

Decorators wrap a function or class to add behavior without changing the callsite. The syntax `@decorator` is sugar for `func = decorator(func)`, extending naturally to parameterized decorators (three nesting levels) and to stacking multiple decorators (applied bottom-up). `functools.wraps` is mandatory in every wrapper to preserve the original's metadata.

Technique	Example	Description
Function decorator	<pre>@staticmethod def f(arg): ... # same as: f = staticmethod(f)</pre>	A function that returns another function, applied with <code>@decorator</code> syntax as sugar for <code>func = decorator(func)</code> .
functools.wraps	<pre>@wraps(f) def wrapper():</pre>	Preserves function metadata (<code>__name__</code> , <code>__doc__</code> , <code>__wrapped__</code>), use inside every wrapper.
Decorator with args	<pre>@deco(arg) def func():</pre>	Returns a decorator, three-level nesting : <code>deco(arg)(func)</code> .
Stacking decorators	<pre>@deco1 @deco2 def f():</pre>	Multiple decorators applied bottom-up : <code>deco1(deco2(f))</code> .
Decorator factory with optional arguments	<pre>def repeat(_func=None, *, n=2): def deco(func): ... return deco if _func is None else deco(_func)</pre>	Lets a decorator be used both as <code>@repeat</code> and <code>@repeat(n=3)</code> by making the function an optional first argument.

Technique	Example	Description
Class decorator	<pre>@deco class MyClass:</pre>	Modifies or replaces a class, the callable receives and returns the class .
lru_cache	<pre>@lru_cache(maxsize=128)</pre>	Memoizes function results with LRU eviction , <code>cache_info()</code> reports hits and misses.
functools.cache	<pre>@cache</pre>	Unbounded memoization , faster than <code>lru_cache(None)</code> as no eviction overhead (Python 3.9+).
functools.cached_property	<pre>@cached_property def value(self):</pre>	Computes property once per instance and caches as a normal attribute, delete attribute to reset (Python 3.8+).
functools.partial	<pre>double = partial(mul, 2)</pre>	Pre-fills arguments, creates specialized callable from a general function.
functools.reduce	<pre>reduce(lambda a,b: a+b, nums, initial=0)</pre>	- Folds a sequence left-to-right into a single value <code>initial</code> also accepts a keyword argument (Python 3.14+).
functools.singledispatch	<pre>@singledispatch def fun(arg): ... @fun.register def _(arg: int): ...</pre>	Turns a function into a generic function that dispatches on the type of its first argument.
functools.total_ordering	<pre>@total_ordering class C:</pre>	Generates all comparison methods from just <code>__eq__</code> and one ordering method (<code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , or <code>__ge__</code>).
functools.Placeholder	<pre>from functools import partial, Placeholder as _ remove = partial(str.replace, _, _, '')</pre>	Marks positional gaps in <code>partial()</code> that callers must fill later (Python 3.14+).

TABLE 19

Context Managers

Context managers, driven by `__enter__` / `__exit__` (or `__aenter__` / `__aexit__` for async), guarantee deterministic resource cleanup even if the block raises. The `contextlib` module makes new managers trivial via the `@contextmanager` generator decorator and packages handy ones like `suppress`, `closing`, and the dynamic-composition `ExitStack`.

Technique	Example	Description
with statement	<pre>with open("f") as f:</pre>	Ensures resource cleanup: calls <code>__enter__</code> and <code>__exit__</code> even if the block raises.
Multiple contexts	<pre>with open("a") as f1, open("b") as f2:</pre>	Opens several resources in one statement, acquired left to right and released right to left .
contextlib.contextmanager	<pre>@contextmanager def ctx(): yield</pre>	Turns a generator function into a context manager: yield separates setup from teardown .
Custom enter / exit	<pre>def __exit__(self, exc_type, exc_val, exc_tb): return False</pre>	- Full protocol implementation: <code>__exit__</code> receives (<code>exc_type</code>, <code>exc_val</code>, <code>exc_tb</code>) returning a true value suppresses the exception
contextlib.suppress	<pre>with suppress(FileNotFoundError):</pre>	Ignores specified exceptions, cleaner than <code>try: ... except: pass</code> .
contextlib.closing	<pre>with closing(obj):</pre>	Calls <code>close()</code> on exit, for objects without a native context manager .
contextlib.redirect_stdout	<pre>with redirect_stdout(io.StringIO()) as f: help(pow)</pre>	Temporarily redirects <code>sys.stdout</code> to another file-like object (<code>redirect_stderr</code> does the same for stderr).

Technique	Example	Description
contextlib.nullcontext	<pre>cm = suppress(Exception) if ignore else nullcontext() with cm: ...</pre>	A no-op context manager, a stand-in when a context manager is optional .
ExitStack	<pre>with ExitStack() as stack:</pre>	Manages a variable number of context managers dynamically.
contextlib.AsyncExitStack	<pre>async with AsyncExitStack() as stack: conn = await stack.enter_async_cont ext(get_connection())</pre>	Async counterpart of <code>ExitStack</code> : combines sync and async context managers via <code>enter_async_context()</code> .
contextlib.asynccontextmanager	<pre>@asynccontextmanager async def get_connection(): conn = await acquire() try: yield conn finally: await release(conn)</pre>	The <code>async with</code> counterpart of <code>@contextmanager</code> , built from an async generator function .
contextlib.chdir	<pre>with.chdir("/tmp"): ...</pre>	- Changes the working directory on enter and restores it on exit not safe for threaded, async or generator code (added in 3.11).

TABLE 20

Standard Library Essentials

The stdlib modules a working developer reaches for daily but that do not fill a table of their own: reading and writing data formats, dates, the OS and process layer, and a few small utility modules.

Concept	Example	Description
json.dumps() / json.loads()	<pre>json.dumps(data, indent=2, default=str) obj = json.loads(text)</pre>	- Serializes/deserializes Python objects to/from JSON text <code>dump()</code> / <code>load()</code> work on file objects directly
csv.DictReader / DictWriter	<pre>with open("f.csv", newline="") as f: for row in csv.DictReader(f): print(row["name"])</pre>	- Reads/writes CSV rows as dicts keyed by the header row - open the file with <code>newline=''</code>
datetime.now() / fromisoformat()	<pre>datetime.now(timezone.utc) datetime.fromisoformat("2024-01-15")</pre>	- Gets the current aware time or parses an ISO 8601 string - Always pass a <code>tz</code> to <code>now()</code> to avoid naive datetimes
timedelta	<pre>datetime.now() + timedelta(days=7, hours=2)</pre>	Expresses a duration in days, seconds and microseconds for datetime arithmetic
zoneinfo.ZoneInfo	<pre>dt.datetime(2020, 10, 31, 12, tzinfo =ZoneInfo("America/Los_Angeles"))</pre>	Attaches an IANA time zone to a datetime, handling DST transitions automatically
os.environ.get()	<pre>token = os.environ.get("API_TOKEN", "")</pre>	Reads an environment variable from the mapping captured at process startup
sys.argv / sys.exit()	<pre>if len(sys.argv) < 2: sys.exit(1)</pre>	<code>argv</code> holds the command-line arguments, <code>argv[0]</code> the script name <code>exit()</code> raises <code>SystemExit</code> with the given status
subprocess.run()	<pre>subprocess.run(["ls", "-l"], check=True, capture_output=True, text=True)</pre>	- Runs an external command and waits for it, returning a <code>CompletedProcess</code> <code>check=True</code> raises on a nonzero exit

Concept	Example	Description
argparse.ArgumentParser	<pre>parser.add_argument("filename") parser.add_argument("-v", action="store_true") args = parser.parse_args()</pre>	Builds a command-line interface: positional arguments, options and flags parsed into a <code>Namespace</code>
hashlib.sha256().hexdigest()	<pre>hashlib.sha256(url.encode()).hexdigest()</pre>	- Computes a deterministic hex digest of bytes <code>sha1()</code> is faster but weaker, both fine for non-security ids
shutil.copy / rmtree / which	<pre>shutil.copy("a.txt", "b.txt") shutil.rmtree("build/") shutil.which("git")</pre>	- Copies a file, deletes a directory tree, or finds an executable on <code>PATH</code> <code>rmtree</code> cannot be undone
secrets vs random	<pre>secrets.token_hex(16) random.choice(items)</pre>	<code>secrets</code> generates cryptographically strong values for tokens and passwords <code>random</code> is for simulation, not security
time.perf_counter()	<pre>start = time.perf_counter() ... elapsed = time.perf_counter() - start</pre>	Highest-resolution monotonic clock for timing code, unaffected by system clock changes
heapq	<pre>heapq.heappush(heap, item) heapq.heappop(heap) heapq.heapify_max(items)</pre>	- Turns a list into a binary min-heap in place; <code>heap[0]</code> is the smallest item 3.14 adds <code>heapify_max</code> / <code>heappush_max</code> / <code>heappop_max</code> for max-heaps
bisect.insort() / bisect_left()	<pre>bisect.insort_left(sorted_list, x) i = bisect.bisect_left(sorted_list, x)</pre>	Locates or inserts into a sorted list in $O(\log n)$ without a full sort
unicodedata.normalize()	<pre>unicodedata.normalize("NFC", name)</pre>	Rewrites a Unicode string into a canonical form (<code>NFC</code> , <code>NFKC</code> , <code>NFD</code> , <code>NFKD</code>) so equivalent-looking strings compare equal
compression.zstd (3.14)	<pre>from compression.zstd import compress, decompress compress(data)</pre>	New in 3.14: Zstandard compression and decompression built into the standard library

TABLE 21

Collections Module

The `collections` module provides specialized container datatypes that extend Python's built-in `dict` , `list` , `set` , and `tuple` . `deque` is the go-to for $O(1)$ queue operations, `OrderedDict` retains its usefulness for LRU implementations despite regular dicts being ordered since 3.7, and `Counter` eliminates a common tally-building loop pattern entirely.

Type	Example	Description
deque	<pre>from collections import deque dq = deque([1,2,3], maxlen=5)</pre>	- Double-ended queue: $O(1)$ append and pop on both ends <code>list.pop(0)</code> is $O(n)$
deque.appendleft() / popleft()	<pre>dq.appendleft(0) dq.popleft()</pre>	Efficient left-side operations , the core advantage over <code>list</code> for queue usage.
deque with maxlen	<pre>deque(maxlen=10)</pre>	Bounded deque: oldest items auto-discarded when full - perfect for sliding windows
defaultdict	<pre>d = defaultdict(list) d["yellow"].append(1)</pre>	- Dict that auto-creates missing keys using a factory function - the classic idiom for grouping values by key
Counter	<pre>Counter("abracadabra")</pre>	Dict subclass for counting hashable objects : <code>most_common(n)</code> returns top-n items.
Counter arithmetic	<pre>c1 + c2 , c1 - c2 , c1 & c2</pre>	<code>+</code> , <code>-</code> , <code>&</code> (intersection) and <code> </code> (union) - output excludes counts of zero or less
Counter.total()	<pre>c.total()</pre>	Returns sum of all counts (Python 3.10+).

Type	Example	Description
namedtuple	<code>Point = namedtuple('Point', 'x y')</code>	Tuple with named fields : immutable, memory-efficient alternative to a class.
typing.NamedTuple	<code>class Employee(NamedTuple): name: str id: int</code>	Class-based, typed alternative to <code>namedtuple()</code> with field annotations.
OrderedDict	<code>OrderedDict()</code>	- Dict that remembers insertion order and supports <code>move_to_end()</code> - still useful for LRU caches
OrderedDict.move_to_end()	<code>od.move_to_end("key")</code>	- Moves key to end (or beginning with <code>last=False</code>) - key primitive for LRU cache implementation
ChainMap	<code>ChainMap(local, global_, builtins)</code>	- Groups multiple dicts: searches sequentially without copying - write goes to first map
UserDict / UserList	<code>class MyDict(UserDict):</code>	Subclass-friendly wrappers around <code>dict</code> / <code>list</code> , safer to subclass than the built-ins directly.

TABLE 22

Itertools Module

`itertools` is Python's grab-bag of iterator-combining building blocks, composable, lazy, and implemented in C for speed. The module covers infinite generators (`count`, `cycle`), combinatorics (`combinations`, `permutations`, `product`), grouping by adjacent runs (`groupby`), windowing (`pairwise`, `batched`), and reduction (`accumulate`).

Function	Example	Description
chain()	<code>chain(it1, it2)</code>	Concatenates iterables: flattens multiple iterables into a single iterator.
chain.from_iterable()	<code>chain.from_iterable(nested)</code>	Flattens one level of nesting lazily, like <code>chain(*nested)</code> without unpacking.
islice()	<code>islice(it, start, stop)</code>	Slices an iterator (no negative indices allowed).
compress()	<code>compress('ABCDEF', [1,0,1,0,1,1])</code>	Filters data by a boolean selector iterable, yields A, C, E, F.
filterfalse()	<code>filterfalse(lambda x: x<5, [1,4,6,3,8])</code>	Yields elements where predicate is false , the opposite of <code>filter()</code> .
starmap()	<code>starmap(pow, [(2,5), (3,2)])</code>	Applies function with arguments unpacked from each tuple in iterable.
pairwise()	<code>pairwise([1, 2, 3])</code>	Returns overlapping pairs of consecutive elements: <code>(1,2)</code> , <code>(2,3)</code> (Python 3.10+).
batched()	<code>batched(range(10), 3)</code>	- Splits iterable into fixed-size tuples, last batch may be shorter (Python 3.12+) <code>strict=True</code> raises <code>ValueError</code> if not evenly divisible (3.13)
groupby()	<code>groupby(items, key=func)</code>	- Groups consecutive elements by key - input must be sorted by the same key first
accumulate()	<code>accumulate(nums, func)</code>	- Running totals or reductions, default is addition - accepts any binary function
takewhile()	<code>takewhile(pred, it)</code>	Yields elements while predicate is true , stopping at the first false.
dropwhile()	<code>dropwhile(pred, it)</code>	Skips elements while predicate is true , then yields all remaining.
tee()	<code>tee(iterable, n=2)</code>	Splits one iterator into n independent iterators , use before the iterator is consumed.

Function	Example	Description
<code>zip_longest()</code>	<code>zip_longest(a, b, fillvalue=None)</code>	Like <code>zip()</code> but continues to the longest input, filling shorter ones with <code>fillvalue</code> .
<code>combinations()</code>	<code>combinations(items, r)</code>	All r-length combinations: order doesn't matter , no repeated elements.
<code>combinations_with_replacement()</code>	<code>combinations_with_replacement('AB', 2)</code>	All r-length combinations allowing repeated elements : yields AA, AB, BB .
<code>permutations()</code>	<code>permutations(items, r)</code>	All r-length permutations: order matters , no repeated elements.
<code>product()</code>	<code>product(a, b)</code>	Cartesian product, equivalent to nested for-loops .
<code>itertools.count()</code>	<code>count(start=0, step=1)</code>	Infinite counter , useful with <code>zip()</code> or <code>takewhile()</code> .
<code>cycle()</code>	<code>cycle([1, 2, 3])</code>	Repeats iterable infinitely , saves a copy of all elements in memory.
<code>repeat()</code>	<code>repeat(10, times=3)</code>	Repeats object <code>n</code> times, infinite if times omitted .

TABLE 23

Regular Expressions

The `re` module exposes a single regex engine through both functional (`re.search` , `re.findall` , `re.sub`) and compiled (`re.compile(...)`) APIs. The biggest pitfalls are `re.match` vs `re.search` (anchored vs unanchored) and forgetting `r"..."` raw strings; without them, Python's own backslash processing eats half your pattern before the engine sees it.

Function	Example	Description
<code>re.search()</code>	<code>re.search(r"\d+", s)</code>	- Searches anywhere in the string - returns the first match object or <code>None</code> .
<code>re.match()</code>	<code>re.match(r"\d+", s)</code>	- Matches only at the start of the string - returns a match object or <code>None</code> .
<code>re.fullmatch()</code>	<code>re.fullmatch(r"\d+", s)</code>	- Matches the entire string - both <code>^</code> and <code>\$</code> are implicit
<code>re.findall()</code>	<code>re.findall(r"\d+", s)</code>	Returns a list of all non-overlapping matches as strings (or tuples if the pattern has groups).
<code>re.finditer()</code>	<code>re.finditer(r"\d+", s)</code>	- Returns an iterator of match objects - memory-efficient for large inputs
<code>re.sub()</code>	<code>re.sub(r"\d+", "X", s)</code>	- Replaces matches - accepts a function as the replacement for dynamic substitutions
<code>re.split()</code>	<code>re.split(r"\s+", s)</code>	- Splits by pattern - captured groups in the pattern are kept in the result list
<code>re.compile()</code>	<code>p = re.compile(r"\d+")</code>	- Precompiles a pattern into a regex object - faster for reuse across many strings
<code>re.escape()</code>	<code>re.escape("https://a.b")</code>	Escapes regex metacharacters in a literal string before building a pattern from it.
<code>Match.group()</code>	<code>m.group(1)</code>	- Returns one or more captured groups <code>group(0)</code> is the whole match, <code>group(1)</code> the first capture
<code>Match.groups()</code>	<code>m.groups()</code>	- Returns a tuple of all subgroups from 1 up - a group that did not participate defaults to <code>None</code> .
<code>Match.span()</code>	<code>m.span()</code>	Returns the (start, end) tuple of the match, or of a given group.
Named groups	<code>(?P<name>\d+)</code>	- Named capture - access via <code>m.group('name')</code> or <code>m.groupdict()</code> .
Non-greedy	<code>r"\d+?"</code>	? after a quantifier matches as few characters as possible.

Function	Example	Description
Lookahead / lookbehind	<code>r"\d+(?= px)"</code>	- Zero-width assertions: <code>(?=...)</code> lookahead, <code>(?<=...)</code> lookbehind - neither consumes characters
re.IGNORECASE	<code>re.search(r"a", s, re.I)</code>	- Case-insensitive matching - Combine flags with <code>re.I</code> <code>re.M</code> .
re.DOTALL	<code>re.search(r".+", s, re.S)</code>	- Makes <code>.</code> match newlines too - useful for multi-line text blocks
re.MULTILINE	<code>re.search(r"^word", s, re.M)</code>	<code>^</code> and <code>\$</code> match the start/end of each line, not just the whole string.
re.VERBOSE	<code>re.compile(r"\d+ # int part\n.\d*", re.X)</code>	Ignores whitespace and <code>#</code> comments in the pattern, for readability.
Raw strings	<code>r"\d+"</code>	Prefix with <code>r</code> to stop Python decoding backslash escapes before <code>re</code> sees the pattern.

TABLE 24

Logging Module

Python's `logging` is a hierarchical system: every logger is a node in a tree rooted at the root logger, and events propagate upward unless `propagate=False` . Application code should always call `logging.getLogger(__name__)` , let libraries configure nothing themselves, and leave root-level configuration to the application entry point.

Technique	Example	Description
basicConfig()	<code>logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")</code>	- Configures the root logger in one call, for simple scripts - must run before the first log call
getLogger()	<code>logger = logging.getLogger(__name__)</code>	- Gets or creates a named logger: hierarchy via dot notation - always use <code>__name__</code>
Log levels	<code>logger.debug("...")</code> <code>logger.info("...")</code> <code>logger.warning("...")</code> <code>logger.error("...")</code> <code>logger.critical("...")</code>	Five levels, DEBUG < INFO < WARNING < ERROR < CRITICAL : only events at or above the set level are emitted.
Deferred message formatting	<code>logger.debug("Message with %s, %s", expensive_func1(), expensive_func2())</code>	- Pass values as %-style args: formatting into the message happens only if the record is emitted - guard costly args with <code>logger.isEnabledFor()</code>
StreamHandler	<code>h = logging.StreamHandler()</code> <code>logger.addHandler(h)</code>	Outputs to stderr (default) or any stream: the standard handler for console output.
FileHandler	<code>h = logging.FileHandler("app.log")</code>	- Writes to a file - use <code>RotatingFileHandler</code> for size-limited rotation
RotatingFileHandler	<code>RotatingFileHandler("app.log", maxBytes=1e6, backupCount=3)</code>	Rotates the log file when it reaches <code>maxBytes</code> , keeping <code>backupCount</code> backups.
Formatter	<code>fmt = Formatter("%(asctime)s - %(name)s - %(message)s")</code> <code>h.setFormatter(fmt)</code>	Controls log record format : standard attributes include <code>asctime</code> , <code>name</code> , <code>levelname</code> , <code>message</code> .
logger.exception()	<code>except Exception:</code> <code>logger.exception("Error")</code>	- Logs at ERROR level plus the full stack trace - use inside <code>except</code> blocks
Propagate	<code>logger.propagate = False</code>	Stops log records bubbling up to parent loggers, avoiding duplicate output.

Technique	Example	Description
extra=	<code>logger.warning("Protocol problem: %s", "connection reset", extra={"clientip": "192.168.0.1"})</code>	• Adds custom fields to the LogRecord for use in the format string • keys must not clash with built-in LogRecord attributes
LoggerAdapter	<code>adapter = LoggerAdapter(logger, { "req_id": id })</code>	Adds contextual data to every log call without modifying the message format.
NullHandler (for libraries)	<code>logging.getLogger(__name__).addHandler(logging.NullHandler())</code>	Library code adds this no-op handler so it never emits a "no handlers found" warning before the application configures logging.
dictConfig / fileConfig	<code>logging.config.dictConfig(cfg)</code>	Configures logging from a dict or file : preferred for production over <code>basicConfig</code> .
QueueHandler / QueueListener	<code>q = queue.Queue() logger.addHandler(QueueHandler(q)) QueueListener(q, StreamHandler()).start()</code>	• Moves logging onto a separate thread via a queue • keeps slow handlers off the app's critical path

TABLE 25

Type Hints

Type hints are runtime metadata that static checkers (`mypy` , `pyright` , `pyre`) consume to find bugs before they ship. PEP 585 (Python 3.9) made built-in generics work directly (`list[int]`), PEP 604 (3.10) introduced `int | None` unions, PEP 695 (3.12) added inline generic-parameter syntax, and PEP 649/749 (3.14) defer annotation evaluation until it is actually needed, cutting the runtime cost of defining annotations.

Technique	Example	Description
Function annotations	<code>def f(x: int) -> str:</code>	Types parameters and the return value, read by checkers such as <code>mypy</code> and <code>pyright</code> but ignored at runtime.
Basic types	<code>x: int = 5</code>	Annotates a variable's type but is not enforced at runtime.
Built-in generics	<code>items: list[int] d: dict[str, int]</code>	Use built-in types directly as generics, no <code>List</code> / <code>Dict</code> imports needed (Python 3.9+).
Union (pipe syntax)	<code>x: int str</code>	Combines multiple allowed types with the modern union operator , replacing <code>Union[int, str]</code> (Python 3.10+).
TypeVar	<code>T = TypeVar('T')</code>	Generic type variable for parametric polymorphism in functions and classes.
Type parameter syntax	<code>def f[T](x: T) -> T:</code>	Inline generic type variables on functions, classes and type aliases, replacing <code>TypeVar</code> (Python 3.12+).
TypeVar defaults	<code>class Cache[T = str]:</code>	Gives a type parameter a default , so callers can omit it (PEP 696, Python 3.13+).
type statement	<code>type Vector = list[float]</code>	Declares a type alias with lazy evaluation, replacing the <code>TypeAlias</code> annotation (Python 3.12+).
ParamSpec	<code>P = ParamSpec('P')</code>	Captures the parameter types of a callable, for typing decorators that preserve signatures.
Unpack	<code>def f(**kwargs: Unpack[Config]):</code>	Types `kwargs` ** using a <code>TypedDict</code> , enabling per-key type checking (Python 3.12+).
Callable	<code>f: Callable[[int], str]</code>	• A list of parameter types, then the return type • <code>collections.abc.Callable</code> is now preferred over the deprecated <code>typing.Callable</code>
Protocol	<code>class Sized(Protocol): def __len__(self): ...</code>	Structural subtyping: duck typing with static type checking.

Technique	Example	Description
TypedDict	<pre>class Config(TypedDict): host: str port: int</pre>	Typed dict with specific key/value types , enabling type checking on dict literals.
Required / NotRequired / ReadOnly	<pre>class Movie(TypedDict): title: str year: NotRequired[int]</pre>	- Mark a single <code>TypedDict</code> key as always present, optional, or read-only <code>ReadOnly</code> arrived in 3.13, the other two in 3.11
Literal	<pre>x: Literal["a", "b"]</pre>	Restricts to specific values , useful for discriminated unions.
Any	<pre>x: Any</pre>	- Unconstrained type, disables checking avoid when possible, an escape hatch
Self	<pre>def add(self) -> Self: return self</pre>	Represents the enclosing class , so subclasses keep the right return type without repeating it (Python 3.11+).
NewType	<pre>UserId = NewType('UserId', int)</pre>	Creates a distinct type for checkers only , though calling it returns the argument unchanged at runtime.
Final and ClassVar	<pre>MAX: Final = 100 class C: count: ClassVar[int] = 0</pre>	<code>Final</code> names cannot be reassigned <code>ClassVar</code> marks an attribute as class-level, not per-instance
cast() and TYPE_CHECKING	<pre>x = cast(int, get_value()) if TYPE_CHECKING: import expensive_module</pre>	<code>cast()</code> tells the checker a value's type without a runtime check <code>TYPE_CHECKING</code> is <code>True</code> only for checkers, guarding import-only-for-typing code
Never / NoReturn	<pre>def stop() -> Never: raise RuntimeError('no way')</pre>	The bottom type : marks a function that never returns normally.
Typels	<pre>def is_str(x) -> TypeIs[str]:</pre>	Narrows the type in both if/else branches, safer than <code>TypeGuard</code> (Python 3.13+).
@override	<pre>@override def method(self):</pre>	Marks a method as overriding a parent , so type checkers flag mismatches (Python 3.12+).
Deferred annotations	<pre>def f(x: MyClass) -> None:</pre>	In Python 3.14+, annotations are not evaluated at import time : forward references work natively without quotes.
annotationlib.get_annotations()	<pre>from annotationlib import get_annotations, Format get_annotations(func, format=Format.FORWARDREF)</pre>	Reads an object's annotations in a chosen format (VALUE, FORWARDREF, STRING), working with 3.14's deferred evaluation (Python 3.14+).
Optional[T] (legacy-style)	<pre>x: Optional[int] = None</pre>	Shorthand for <code>Union[T, None]</code> , though new code prefers the union-pipe form instead.

TABLE 26

Async/Await (Asyncio)

`asyncio` delivers cooperative single-threaded concurrency: many waiting coroutines, one running at a time, no blocking. `await` is the only yield point (every awaited call must be non-blocking), and Python 3.11+ adds *structured concurrency* via `TaskGroup` plus deadline-based `timeout()`. The synchronization primitives (`Lock`, `Event`, `Semaphore`, `Queue`) mirror the `threading` module but are awaitable, not blocking.

Technique	Example	Description
async def	<pre>async def fetch():</pre>	Defines a coroutine function, must be awaited or scheduled as a task.
await	<pre>result = await coro()</pre>	Suspends the coroutine, yielding control to the event loop until the result is ready.
asyncio.run()	<pre>asyncio.run(main())</pre>	- Entry point: creates the event loop and runs the coroutine - closes the loop on completion

Technique	Example	Description
asyncio.create_task()	<code>task = create_task(coro())</code>	Schedules a coroutine to run concurrently with other tasks.
Task references	<code>background_tasks = set() task = create_task(coro()) background_tasks.add(task) task.add_done_callback(background_tasks.discard)</code>	- The event loop keeps only a weak reference to a task - hold a strong reference or it can be garbage collected mid-run
asyncio.TaskGroup	<code>async with TaskGroup() as tg: tg.create_task(coro())</code>	Structured concurrency: auto-cancels remaining tasks on failure (Python 3.11+).
asyncio.gather()	<code>await gather(coro1(), coro2())</code>	Runs multiple coroutines concurrently, waits for all to complete.
Task cancellation	<code>task.cancel() try: await task except asyncio.CancelledError: cleanup()</code>	<code>cancel()</code> requests cancellation; <code>CancelledError</code> is raised inside the task - catch it to clean up, then let it propagate
asyncio.timeout()	<code>async with asyncio.timeout(5):</code>	Context manager with a deadline , raises <code>TimeoutError</code> if exceeded (Python 3.11+).
asyncio.wait_for()	<code>await wait_for(coro(), timeout=5)</code>	Adds a timeout to a coroutine, raises <code>TimeoutError</code> if exceeded.
asyncio.to_thread()	<code>await asyncio.to_thread(blocking_io)</code>	Runs a blocking, synchronous function in a separate thread without blocking the event loop.
asyncio.Lock	<code>async with asyncio.Lock():</code>	Mutex lock for async tasks, prevents concurrent access to shared state.
asyncio.Event	<code>event = asyncio.Event() await event.wait()</code>	Notifies multiple tasks that something happened: <code>set()</code> , <code>clear()</code> , <code>wait()</code> .
asyncio.Semaphore	<code>async with asyncio.Semaphore(10):</code>	Limits number of concurrent coroutines accessing a resource.
asyncio.Queue	<code>q = asyncio.Queue() await q.put(x) x = await q.get()</code>	FIFO queue for producer-consumer patterns: <code>task_done()</code> plus <code>join()</code> for completion tracking.
asyncio.sleep()	<code>await asyncio.sleep(1)</code>	Non-blocking sleep, yields control to other coroutines.
async for	<code>async for item in async_iter:</code>	Iterates over an async iterable , calling <code>__aiter__</code> and <code>__anext__</code> .
async with	<code>async with resource:</code>	Async context manager, awaits <code>__aenter__</code> and <code>__aexit__</code> .
asyncio.Runner	<code>with asyncio.Runner() as runner: runner.run(main())</code>	- Reusable context manager for running several coroutines in the same event loop and context (Python 3.11+) <code>asyncio.run()</code> covers most single-call cases
asyncio introspection CLI	<code>python -m asyncio ps PID python -m asyncio pstree PID</code>	- Inspects a running process's asyncio tasks (Python 3.14+) <code>ps</code> lists tasks, <code>pstree</code> renders the call tree for debugging stuck programs

TABLE 27

Concurrency

Python offers three concurrency models: **threading** (shared memory, GIL-limited for CPU work, good for I/O), **multiprocessing** (separate processes, true parallelism, higher overhead), and **asyncio** (cooperative single-thread, ultra-low overhead, requires async-aware code). `concurrent.futures` wraps both threading and multiprocessing behind one uniform executor interface; Python 3.14 adds `InterpreterPoolExecutor` for sub-interpreter concurrency without the IPC overhead of full processes.

Technique	Example	Description
ThreadPoolExecutor	<code>with ThreadPoolExecutor(max_workers=4) as ex: fut = ex.submit(func, arg)</code>	- Thread pool: best for I/O-bound tasks - GIL limits CPU parallelism

Technique	Example	Description
ProcessPoolExecutor	<pre>with ProcessPoolExecutor() as ex: fut = ex.submit(func, arg)</pre>	- Process pool: true CPU parallelism - data is pickled between processes
executor.map()	<pre>list(ex.map(func, items))</pre>	Parallel map, returns results in submission order and raises the first exception encountered.
Future.result()	<pre>fut.result(timeout=5)</pre>	Blocks until the result is available, re-raises any exception thrown by the worker.
as_completed()	<pre>for f in as_completed(futures):</pre>	Yields futures as they complete , better throughput than waiting in submission order.
Executor.shutdown()	<pre>ex.shutdown(wait=True, cancel_futures=True)</pre>	- Frees executor resources once pending futures finish <code>cancel_futures=True</code> cancels futures that have not started running yet
threading.Thread	<pre>t = Thread(target=f, args=(1,)) t.start(); t.join()</pre>	- Spawns a native OS thread - use <code>daemon=True</code> to die with the main thread
threading.Lock	<pre>lock = Lock() with lock:</pre>	- Mutex for shared state <code>with lock:</code> is preferred (auto-releases on exceptions).
threading.Event	<pre>e = Event() e.set(); e.wait()</pre>	Lets threads signal each other : <code>wait()</code> blocks until <code>set()</code> is called.
threading.Semaphore	<pre>sem = Semaphore(3)</pre>	Limits concurrent access to <code>n</code> threads: <code>acquire()</code> decrements, <code>release()</code> increments.
threading.local	<pre>data = threading.local() data.value = 42</pre>	Per-thread storage: each thread sees its own attribute values on the same object.
queue.Queue	<pre>q = queue.Queue() q.put(item) q.get()</pre>	- Thread-safe FIFO for handing work between threads <code>put()</code> / <code>get()</code> block by default until a free slot or an item is available
multiprocessing.Pool	<pre>with Pool() as pool: pool.map(func, items)</pre>	- Higher-level multiprocessing API automatically manages worker process lifecycle
multiprocessing.Queue	<pre>q = mp.Queue() q.put(x); q.get()</pre>	- Process-safe queue IPC via pickle across process boundaries
if name == 'main'	<pre>if __name__ == '__main__': p = Process(target=f) p.start()</pre>	- Protects process creation from re-running on import - without it, the spawn/forkserver methods re-import the main module and raise <code>RuntimeError</code>.
Start methods	<pre>mp.set_start_method('spawn') ctx = mp.get_context('forkserver')</pre>	spawn: default on Windows/macOS, slower but safest forkserver: default on other POSIX platforms since 3.14 fork: POSIX only, unsafe with threads
InterpreterPoolExecutor	<pre>with InterpreterPoolExecutor() as ex:</pre>	Runs tasks in sub-interpreters : true parallelism without pickle overhead (Python 3.14+).
concurrent.interpreters	<pre>import concurrent.interpreters</pre>	New in Python 3.14 (PEP 734): runs code in isolated sub-interpreters with true parallelism via <code>InterpreterPoolExecutor</code> .
GIL and free-threading	<pre>python3.14 -X gil=0 sys._is_gil_enabled()</pre>	- The free-threaded build is officially supported but still optional (3.14, phase II) - check its status at runtime with <code>sys._is_gil_enabled()</code>.

References

Official Documentation

1. [int](#)
3. [Decimal / Fraction](#)
5. [Augmented assignment](#)
7. [f-strings](#)
9. [Basic slice](#)
11. [del statement](#)
13. [if](#)
15. [Return multiple values as a tuple](#)
17. [Generator pipeline](#)
19. [try-except](#)
21. [ExceptionGroup](#)
23. [pathlib.Path](#)
25. [import module](#)
27. [python -m module](#)
29. [importlib](#)
31. [uv init](#)
33. [poetry](#)
35. [ABC / @abstractmethod](#)
37. [@dataclass](#)
39. [Enum](#)
41. [functools.wraps](#)
43. [json.dumps\(\) / json.loads\(\)](#)
45. [datetime.now\(\) / fromisoformat\(\)](#)
47. [os.environ.get\(\)](#)
49. [argparse.ArgumentParser](#)
51. [shutil.copy / rmtree / which](#)
53. [time.perf_counter\(\)](#)
55. [bisect.insort\(\) / bisect_left\(\)](#)
57. [compression.zstd \(3.14\)](#)
59. [typing.NamedTuple](#)
61. [basicConfig\(\)](#)
63. [StreamHandler](#)
65. [annotationlib.get_annotations\(\)](#)
67. [asyncio.run\(\)](#)
69. [asyncio.Queue](#)
71. [ThreadPoolExecutor](#)
73. [queue.Queue](#)
75. [concurrent.interpreters](#)
77. [Python 3 Official Documentation](#)
79. [Python 3.12 What's New](#)
81. [Python 3.10 What's New](#)
83. [asyncio — Asynchronous I/O](#)
85. [Sorting HOWTO](#)
87. [Regex HOWTO](#)
89. [Pydantic Documentation](#)
91. [pytest Documentation](#)
93. [Learn X in Y Minutes — Python](#)
2. [Mutable vs Immutable](#)
4. [Arithmetic](#)
6. [Identity](#)
8. [Format spec mini-language](#)
10. [slice\(\) objects](#)
12. [Shallow copy](#)
14. [while loop](#)
16. [Global and nonlocal scope](#)
18. [Generator send\(\) / close\(\)](#)
20. [Built-in exception hierarchy](#)
22. [read\(\)](#)
24. [tempfile.NamedTemporaryFile](#)
26. [if name == "main"](#)
28. [sys.path](#)
30. [venv create](#)
32. [pyproject.toml](#)
34. [init](#)
36. [Descriptor protocol](#)
38. [Pydantic](#)
40. [Enum lookup](#)
42. [contextlib.contextmanager](#)
44. [csv.DictReader / DictWriter](#)
46. [zoneinfo.ZoneInfo](#)
48. [subprocess.run\(\)](#)
50. [hashlib.sha256\(\).hexdigest\(\)](#)
52. [secrets vs random](#)
54. [heapq](#)
56. [unicodedata.normalize\(\)](#)
58. [deque](#)
60. [re.search\(\)](#)
62. [Deferred message formatting](#)
64. [dictConfig / fileConfig](#)
66. [async def](#)
68. [asyncio.Lock](#)
70. [asyncio introspection CLI](#)
72. [threading.Thread](#)
74. [multiprocessing.Pool](#)
76. [GIL and free-threading](#)
78. [Python 3.13 What's New](#)
80. [Python 3.11 What's New](#)
82. [Python Language Reference](#)
84. [Python HOWTOs](#)
86. [Logging Cookbook](#)
88. [Astral uv Documentation](#)
90. [Ruff — Fast Python linter](#)
92. [Python unittest Documentation](#)

Specifications & Standards

94. [Walrus operator](#)
96. [t-strings](#)
98. [yield from](#)
100. [Deferred annotations](#)
102. [PEP 20 — The Zen of Python](#)
104. [PEP 526 — Syntax for Variable Annotations](#)
106. [PEP 557 — Data Classes](#)
95. [Dict merge operator](#)
97. [Docstring](#)
99. [Type parameter syntax](#)
101. [PEP 8 — Style Guide for Python Code](#)
103. [PEP 484 — Type Hints](#)
105. [PEP 544 — Protocols: Structural subtyping](#)
107. [PEP 585 — Type Hinting Generics in Standard Collections](#)

- 108. PEP 604 — Allow X | Y syntax for unions
- 110. PEP 635 — Structural Pattern Matching Motivation
- 112. PEP 646 — Variadic Generics
- 114. PEP 698 — Override decorator for static typing
- 116. PEP 719 — Python 3.13 Release Schedule
- 118. PEP 734 — Multiple Interpreters in the stdlib
- 120. PEP 3107 — Function Annotations
- 109. PEP 634 — Structural Pattern Matching
- 111. PEP 636 — Structural Pattern Matching Tutorial
- 113. PEP 649 — Deferred Evaluation of Annotations
- 115. PEP 703 — Making the Global Interpreter Lock Optional
- 117. PEP 729 — Typing governance process
- 119. PEP 779 — Criteria for supported status for free-threaded Python
- 121. OWASP Python Security

Books

- 122. Fluent Python 2nd Edition (O'Reilly)
- 123. Python Testing with pytest (Brian Okken)
- 124. Clean Code in Python (Mariano Anaya)
- 125. High Performance Python 2nd Edition (O'Reilly)
- 126. Python Cookbook 3rd Edition (O'Reilly)
- 127. Python One-Liners (Mayer)

Source Code & Repositories

- 128. pyenv
- 130. Python Type Stubs (typeshed)
- 132. pydantic (GitHub)
- 134. astral-sh/uv (GitHub)
- 136. poetry (GitHub)
- 138. pendulum (GitHub)
- 140. httpx (GitHub)
- 142. Python type_extensions (GitHub)
- 144. Python patterns (GitHub)
- 146. 30-seconds-of-python (GitHub)
- 148. CPython issue tracker (GitHub Issues)
- 150. pyperf — Python benchmarking toolkit
- 152. Python Cheat Sheet Community (gto76)
- 129. CPython source code (GitHub)
- 131. mypy (GitHub)
- 133. attrs (GitHub)
- 135. astral-sh/ruff (GitHub)
- 137. structlog (GitHub)
- 139. Rich — terminal output library (GitHub)
- 141. aiohttp (GitHub)
- 143. awesome-python (GitHub)
- 145. wtfpython — Python quirks explained (GitHub)
- 147. Python Package Index (PyPI)
- 149. Python Benchmarks (pyperformance)
- 151. Python REPL improvements (ptpython)

Articles & Tutorials

- 153. pip install
- 155. pip install -e (editable)
- 157. Singleton pattern
- 159. Decorator with args
- 161. Python Packaging User Guide
- 163. Real Python — Python Type Checking Guide
- 165. Real Python — Python Metaclasses
- 167. Real Python — Python Concurrency Guide
- 169. Real Python — Regular Expressions in Python
- 171. Real Python — Python Generators Guide
- 173. Real Python — Python Enum Guide
- 175. Real Python — Python functools Module
- 177. Real Python — Python Scope and LEGB Rule
- 179. Real Python — Structural Pattern Matching in Python 3.10
- 181. Real Python — Python Walrus Operator
- 183. Real Python — Python 3.14 Preview
- 185. mypy Documentation
- 187. Ben Hoyt — Pattern Matching in Python 3.10
- 189. OpenSourceForu — Python 3.13 and 3.14 Features
- 191. Python Speed — Profiling Python Programs
- 193. tonybaloney.github.io — Python 3.13 free-threading
- 195. Brett Cannon — How importlib.resources works
- 197. Python Tricks: The Book (Dan Bader)
- 199. Twilio Blog — Python Concurrency Comparison
- 201. Python Engineer — Python 3.12 new features
- 203. Martin Heinz — Python Project Setup in 2024
- 205. Python Discord Community
- 154. python -m pip
- 156. pip freeze
- 158. attrs
- 160. pip documentation
- 162. Python Type System Reference
- 164. Real Python — Python Data Classes
- 166. Real Python — Async IO in Python
- 168. Real Python — Python Virtual Environments Guide
- 170. Real Python — Python Logging Guide
- 172. Real Python — Python Itertools Guide
- 174. Real Python — Python collections Module
- 176. Real Python — Python Context Managers
- 178. Real Python — Python String Formatting Best Practices
- 180. Real Python — Python 3.12 f-strings
- 182. Real Python — Python Annotations
- 184. attrs Documentation
- 186. pyright Documentation
- 188. Cloudsmith — Python 3.14 Changes
- 190. Hynek Schlawack — Structlog (Python logging)
- 192. Python Tips — Generators and Itertools
- 194. Itamar Turner-Trauring — Getting full value from Python type hints
- 196. Effective Python 3rd Edition (Addison-Wesley)
- 198. Python 3 Patterns, Recipes and Idioms (Bruce Eckel)
- 200. Cloudflare Blog — Python async patterns
- 202. Corey Schafer Python Tutorials
- 204. Arjan Codes — Python Design Patterns
- 206. Talk Python to Me Podcast

- 207. Python Bytes Podcast
- 209. Python Best Practices for Code Quality (Wemake)
- 211. isort — Import sorting
- 213. Safety — Dependency security scanner
- 215. CPython Internals (Anthony Shaw)
- 217. pre-commit hooks
- 219. PyCoder's Weekly
- 221. Awesome Python — curated list of frameworks and tools
- 223. Python Wiki
- 225. Jupyter Project Documentation
- 227. Devhints Python Cheat Sheet

Community Q&A

- 228. Stack Overflow — Python tag
- 229. Python subreddit
- 230. learnpython subreddit

Video

- 231. mCoding — Python Concurrency (YouTube)
- 233. Corey Schafer — Python Decorators (YouTube)
- 235. Arjan Codes — Python Design Patterns (YouTube)
- 237. PyCon US 2025 (YouTube)
- 239. David Beazley — Python Concurrency from the Ground Up (PyCon 2015)
- 241. James Powell — So you want to be a Python expert (PyData 2017)
- 208. Google Python Style Guide
- 210. Black Code Formatter
- 212. Bandit — Security linter for Python
- 214. Architecture Patterns with Python (Harry Percival & Bob Gregory)
- 216. coverage.py Documentation
- 218. Python Weekly Newsletter
- 220. Import Python Newsletter
- 222. Python Discuss (discuss.python.org)
- 224. Python Speed Center
- 226. IPython Documentation
- 232. mCoding — Python Type Hints (YouTube)
- 234. Corey Schafer — Python OOP Tutorials (YouTube)
- 236. PyCon US 2024 Talks (YouTube)
- 238. Raymond Hettinger — Beyond PEP 8 (PyCon 2015)
- 240. Brett Slatkin — Refactoring Python (PyCon 2016)
- 242. Ned Batchelder — Facts and Myths about Python names and values